武漢大學

本科毕业论文（设计）

基于共享内存事件格式的嵌入式二进制日志系统设计与实现

姓　　名：邓泽宇

学　　号：2022302111098

专　　业：软件工程

学　　院：计算机学院

指导教师：武小平

二〇二六年三月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：_____　　指导教师签名：_____

日　期：_____

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：_____　　指导教师签名：_____

日　期：_____

摘　要

面向嵌入式与边缘设备，日志系统既要保证可观测性，又要控制存储、CPU 与并发初始化开销。本文提出并实现 Optbinlog：采用事件格式描述文件、共享元数据和二进制编码路径，将格式解析成本从热路径迁出，并通过偏移量访问保证多进程映射一致性。

本文首先调查国内外代表性日志系统（Linux ftrace、LTTng、Zephyr logging、systemd-journald、NanoLog、RT-Thread ulog、OpenHarmony HiLog Lite）及其优化代码路径，再与 Optbinlog 在算法复杂度、数据结构、并发行为和工程适配性上进行比较。随后基于理论推导与实验验证相结合的方法，分析不同日志模式下日志条数 N、设备数 D 与时间、空间、吞吐之间的关系。

本文采用的保留数据集包括三类：2026-03-09 全分组语义对齐矩阵（工程近似层）、2026-03-12 严格语义对齐套件（single/multi/模拟真实多设备（多台虚拟机节点））以及本地/节点级初始化竞争结果。当前 Optbinlog 采用“共享 schema/tag cache + 每条记录 CRC32C 硬件校验 + string-heavy 场景自动启用 varstr”的实现路径。结果表明，binary 相对 text 的性能收益仍具有显著语义分组依赖；在严格公平口径下，Optbinlog 已在单机、本地多设备和真实多设备三层场景中稳定领先 `ulog/hilog` 两组 peer 语义路径，在 `nanolog` 组呈现“单机可取得优势、真实多设备略回落”的边界行为，而在 `zephyr deferred` 这类批处理紧凑路径上仍保留时间劣势。消融结果进一步表明，当前剩余差异主要来自 `cold-path` 固定成本、通用打包路径以及在“新进程 + 节点并发”场景下难以完全释放的缓存稳态收益。初始化竞争实验稳定满足“单创建者、其余进程/节点复用已初始化共享文件”的正确性约束。理论推导与实测总体一致：空间随 N 线性增长，吞吐随 D 增长呈现饱和与回落特征。

关键词：嵌入式日志；二进制编码；共享内存；并发初始化；性能建模

ABSTRACT

For embedded and edge systems, logging must balance observability with overhead (storage, CPU, and concurrent initialization). This thesis implements Optbinlog, a schema-shared binary logging approach for embedded deployment. Event metadata are parsed once, materialized in shared memory, and accessed via offsets, so runtime hot-path cost is reduced and cross-process mapping remains stable.

The work first surveys representative domestic and international logging systems (Linux ftrace, LTTng, Zephyr logging, systemd-journald, NanoLog, RT-Thread ulog, OpenHarmony HiLog Lite) and analyzes their optimization code paths. Then a theory-first model is built to derive how time/space/throughput scale with log count `N` and device count `D`. Finally, measured results are compared against the derived trends.

The retained dataset combines the 2026-03-09 grouped semantic matrix rerun (engineering-approximate layer), the 2026-03-12 strict-aligned suite (single, local multi-device, and real multi-node simulation), and the local/node-level initialization competition results. The current Optbinlog implementation uses shared schema/tag caching, per-record CRC32C with hardware acceleration, and automatic varstr encoding on string-heavy schemas. Results still show strong semantic-group dependency, but under the strict-aligned setting the binary path remains clearly ahead of the `ulog` and `hilog` peer-style paths across all three scenarios, shows boundary behavior on `nanolog` (ahead on a single node, slightly behind in the real multi-node run), and continues to trail the `zephyr deferred`-style compact batch path. Ablation results further indicate that the dominant remaining overhead now lies in cold-path setup, generic packing work, and the limited cache reuse exposed by fresh-process concurrent runs. Initialization experiments consistently satisfy the single-creator property (one creator, other processes/nodes open existing shared metadata). Theoretical trends and empirical results are broadly aligned, and semantic-equivalent comparison remains a strict requirement for fair cross-system conclusions.

Keywords: embedded logging; binary encoding; shared metadata; concurrency; performance modeling

目录

摘要 I

ABSTRACT II

1 绪论 1

1.1 研究背景 1

1.2 研究问题 1

1.3 本文工作 1

1.4 研究方法与工作中心 1

1.5 研究创新点 2

2 国内外研究调查与缺陷分析 2

2.1 调研范围与方法 2

2.2 国际代表系统与优化思路（代码级） 2

2.3 国内生态系统与优化思路（代码级） 4

2.4 Optbinlog 与六类 peer 系统的对比 4

2.5 使用场景与工作中心差异 5

2.6 典型优化代码入口（可追溯） 6

2.7 现有方案缺陷归纳 6

2.8 Optbinlog 对缺陷的对应关系 7

2.9 复现范围与可行性说明（2026-03-05） 7

3 Optbinlog 设计迭代与工程实现 7

3.1 设计目标 7

3.2 事件格式与元数据结构 8

3.3 写入/读取路径 8

3.4 关键迭代 9

3.5 与实际场景的匹配性 10

4 理论模型与数学推导 10

4.1 符号定义 10

4.2 从算法与数据结构到单条代价 11

4.3 空间随 N 的详细推导 13

4.4 时间随 N 的详细推导 15
4.5 多设备 D 的详细推导 16
4.6 模式间理论排序与可检验预测 18
4.7 与第 5 章指标的映射关系 19

5 实验迭代、对比与理论一致性分析 19
5.1 实验组织与迭代逻辑 19
5.2 单机高负载：严格语义对齐结果 20
5.3 本地多设备模拟：竞争项放大后的趋势 22
5.4 模拟真实多设备（多台虚拟机节点）并发：严格语义对齐结果 . . 25
5.5 初始化竞争说明 27
5.6 理论推导与实测对照 28
5.7 原因分析与消融佐证 28
5.8 实验数据组织与证据链 30
5.9 差异点原因与 Optbinlog 应用优势 30

6 结论与展望 31
6.1 工作总结 31
6.2 当前局限 31
6.3 下一步优化计划 32

参考文献 33

致谢 35

附录 A 关键复现实验命令（当前保留结果） 35

1 绪论

1.1 研究背景

嵌入式系统日志常处于“高频写入、有限存储、多进程并发、实时约束”并存的环境。传统文本日志在可读性上有优势，但会带来：

1. 字符串格式化与解析开销高。
2. 单条日志冗余字节多，长期空间成本高。
3. 多进程共享格式信息时容易出现初始化竞争和一致性问题。

因此，本文并不局限于 `text` 与 `binary` 的局部比较，而是将问题置于“理论分析、实现机制与分组对照实验”统一框架下进行讨论。

1.2 研究问题

1. 现有日志系统的主流优化点是什么？是否仍有缺口？
2. Optbinlog 的实现路径能否覆盖这些缺口？
3. 从算法与数据结构出发，N 与 D 的时间/空间/吞吐趋势应如何推导？
4. 推导结论与实验结果哪些一致，哪些不一致，原因是什么？

1.3 本文工作

1. 完成国内外日志系统调研，并定位代表性优化代码入口。
2. 设计并实现 Optbinlog（事件格式解析、共享元数据、二进制读写、并发初始化状态机）。
3. 建立抽象数学模型，推导不同模式在N、D 维度下的性能趋势。
4. 组织多轮实验（单机高负载、多设备模拟、模拟真实多设备（多台虚拟机节点）并发、init-race）并进行理论对照分析。

1.4 研究方法与工作中心

围绕前述研究问题，本文将工作中心放在三条主线上：1. **机制主线**：从日志系统算法与数据结构出发，不先下结论，先解释为什么会快/慢、为什么会大/小。2. **对照主线**：不只做 `text` 对比，还和 `syslog/ftrace/nanolog/zephyr/ulog/hilog` 做同语义分组对照。

3. **验证主线**：统一统计口径与可复现实验脚本，按“理论推导 -> 单机 -> 多设备 -> 模拟真实多设备（多台虚拟机节点）”逐层验证。

1.5 研究创新点

1. 提出“共享 schema + 偏移访问”的用户态二进制日志实现，重点解决多进程初始化一致性与重复解析问题。
2. 从算法与数据结构角度推导 N 与 D 对时间、空间和吞吐的影响，并给出可检验交叉点与饱和点表达式。
3. 建立“工程近似层 + 严格公平层”两级实验体系：前者保留真实系统路径差异，后者统一 schema、统一生成值与统一平台，避免把语义体量差或输入来源差误判为实现优劣。
4. 在模拟真实多设备（多台虚拟机节点）场景下验证初始化竞争与并发写入行为，形成可复现实验流程。

---

2 国内外研究调查与缺陷分析

2.1 调研范围与方法

本章基于官方文档、论文与代码仓库进行调研，聚焦“热路径优化机制”和“可复现实现入口”。

调研对象：1. Linux ftrace / kernel ring buffer。2. LTTng。3. Zephyr logging subsystem。4. systemd-journald。5. NanoLog（USENIX ATC 2018）。6. RT-Thread ulog。7. OpenHarmony HiLog Lite。

2.2 国际代表系统与优化思路（代码级）

2.2.1 Linux ftrace（内核追踪）

代码与文档显示其核心是 `per-CPU lockless ring buffer`。在 `ring_buffer.c` 中，写入路径围绕“预留 -> 填充 -> 提交”组织，关键是把竞争局部化到 CPU 本地缓冲，降低全局锁竞争。

1. 数据结构：每 CPU buffer + page 链式组织，维护 head/commit/tail。

2. 算法逻辑：优先本地写入、批量可见性提交，读写阶段解耦。
3. 场景定位：内核 tracing、高并发诊断；功能强但接入复杂度高于应用层日志库。

2.2.2 LTTng（Linux Tracing Toolkit Next Generation）

LTTng 采用 tracepoint instrumentation + lockless buffer 管线，强调高吞吐追踪和离线分析。

1. 数据结构：channel/sub-buffer 等追踪缓冲结构。
2. 算法逻辑：将事件采集与后处理分层，追踪点开销可控。
3. 场景定位：系统级性能分析与可观测平台，部署和运维要求较高。

2.2.3 Zephyr logging（RTOS 场景）

在 Zephyr `Kconfig.mode` 与 `log_msg.c` 可见，关键机制是 deferred logging。

1. 数据结构：`log_msg` 对象 + 消息池（MPSC packet buffer）。
2. 算法逻辑：前台线程只做封包入队，后台线程统一输出；支持 overflow/drop 策略。
3. 场景定位：RTOS 中实时任务优先，要求前台日志路径尽量短、可控。

2.2.4 systemd-journald

其 `JOURNAL_FILE_FORMAT` 与 `journald-server` 路径强调二进制对象与索引。

1. 数据结构：header + objects + 哈希/索引。
2. 算法逻辑：写入与检索并重，保证一致性与查询能力。
3. 场景定位：通用 Linux 系统服务日志；对轻量嵌入式而言依赖偏重。

2.2.5 NanoLog（研究型高性能日志）

从 `NanoLogCpp17.h`、`RuntimeLogger.cc` 可见，NanoLog 核心是“编译期提取格式 + 运行期仅记录动态参数”。

1. 数据结构：staging buffer、runtime logger 队列/缓冲。

2. 算法逻辑：前台路径尽量只做参数打包，压缩与格式恢复转移到后台/离线。
3. 场景定位：超高频日志、允许离线恢复文本的性能敏感场景。

2.3 国内生态系统与优化思路（代码级）

2.3.1 RT-Thread ulog

`ulog.c` 提供异步输出和多 backend 抽象，思路是“前台快写缓冲，后台慢速输出”。

1. 数据结构：ringbuffer/rbb、backend 列表。
2. 算法逻辑：异步线程统一调度输出，减轻业务线程阻塞。
3. 场景定位：RT-Thread 设备侧调试与在线日志。

2.3.2 OpenHarmony HiLog Lite

HiLog Lite 在格式能力上做约束（减少复杂格式），换取更低资源开销。

1. 数据结构：轻量日志缓冲和简化格式存储。
2. 算法逻辑：限制格式复杂度，降低格式化路径成本。
3. 场景定位：资源受限设备的系统日志基础设施。

2.4 Optbinlog 与六类 peer 系统的对比

表 2-1 Optbinlog 与六类 peer 系统对比

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
syslog/journald	文本日志 + 系统日志守护进程	通用 Linux 服务运维	生态成熟、检索与管理工具丰富	文本格式化与系统路径开销较高	Optbinlog 在高频写入场景更强调紧凑编码与可控开销

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
ftrace	内核 tracing ring buffer	内核/驱动诊断、时序追踪	内核路径短、追踪能力强	接入复杂、与业务日志模型存在语义差异	Optbinlog 适合业务日志结构化记录，ftrace 适合底层追踪
nanolog_like	固定字段二进制直写	超高频、可离线恢复文本	调用点开销极低	语义扩展与工具链耦合强	Optbinlog 可扩展性更强，但热路径常数项更高
zephyr_like	deferred 入队 + 后台批刷	RTOS 实时任务日志	前台阻塞小	后台线程调度与队列竞争	Optbinlog 不依赖后台线程，行为更确定
ulog_async	异步队列 + 多 backend	嵌入式设备在线调试	工程集成灵活	后台文本格式化与后端竞争	Optbinlog 在结构化与一致性方面更强
hilog_lite_like	受限字段 + 轻量头	资源受限系统日志	资源占用较低	语义表达能力受限	Optbinlog 提供更完整字段语义和校验能力
Optbinlog	共享 schema + typed 编码 + CRC	多进程边缘网关/控制器	跨进程一致性、结构化可扩展、空间可控	初始化和通用编码固定成本	作为“语义完整 + 工程可迁移”的折中方案

2.5 使用场景与工作中心差异

如表 ~2-1 所示，Optbinlog 在设计上更偏向“语义完整、跨进程一致和工程可迁移”的折中路线，因此与追求极限热路径或系统级 tracing 的方案形成了明显分工。

1. 若目标是**内核态追踪**，`ftrace` 仍是优先选择；若目标是**应用层结构化日志**，Optbinlog 更匹配。

2. 若目标是**极限热路径时间**，`nanolog_like/zephyr_like` 这类固定路径模式更有优势；若目标是**语义可扩展与跨进程一致性**，Optbinlog 更稳定。
3. 若系统强调**运维生态兼容**，`syslog/journald` 优势明显；若系统强调**嵌入式长期在线与空间效率**，Optbinlog 更具针对性。

2.6 典型优化代码入口（可追溯）

表 2-2 典型优化代码入口与定位

系统	关键优化点	代码/文档入口
Linux ftrace	每 CPU 无锁 ring buffer	`ring_buffer.c`，设计文档 `ring-buffer-design`
LTTng	tracepoint + lockless buffer	LTTng Concepts 官方文档
Zephyr logging	deferred + 消息池	`Kconfig.mode`、`log_msg.c`
systemd-journald	二进制对象与索引文件	`journald-server.c`、`JOURNAL_FILE_FORMAT`
NanoLog	编译期格式抽取 + 运行时二进制参数	`NanoLogCpp17.h`、`RuntimeLogger.cc`
RT-Thread ulog	异步输出与多 backend	`ulog.c` + ulog 文档
OpenHarmony HiLog Lite	受限格式降低开销	`hiviewdfx_hilog_lite` 文档/代码

2.7 现有方案缺陷归纳

从嵌入式角度可抽象出四类常见缺口：

1. **格式化开销仍在热路径**：text/syslog 在高频场景下受字符串格式化与内核接口调用影响明显。
2. **元数据复用不足**：多进程系统若每进程重复解析格式，启动和内存成本上升。
3. **并发初始化一致性风险**：共享资源初次创建缺少状态机时，容易出现竞态。

4. 跨平台复现实验缺失：只在单平台得到的结论容易失真，外推边界较弱。

2.8 Optbinlog 对缺陷的对应关系

1. 用 “事件格式文件 + 二进制编码” 替代热路径字符串格式化。
2. 用共享元数据文件持久化 tag/element 描述，进程间偏移复用。
3. 用 `INITIALIZING -> INITIALIZED` 状态机控制并发初始化。
4. 用统一脚本输出 mean/median/p95/std/95%CI 与 IQR 过滤结果，支持多轮迭代对比。

2.9 复现范围与可行性说明（2026-03-05）

本文按照 “代码阅读先行、实验复现跟进” 的顺序开展验证工作：

1. 已完成官方代码级阅读：ftrace、Zephyr、NanoLog、ulog 等核心实现入口。
2. 受网络与平台限制：无法完整拉取/编译全部外部系统并在本机跑 Linux tracing 全栈。
3. 已执行算法级复现：实现 `nanolog_like`、`zephyr_deferred_like`、`ulog_async_like`、`hilog_lite_like` 及 “语义对齐” 版本（第 5 章给出数据）。

上述约束已在第 5 章实验方法中显式记录，以区分 “工程可复现边界” 与 “算法层结论边界”。

---

3 Optbinlog 设计迭代与工程实现

3.1 设计目标

1. 降低单条日志热路径开销（时间）。
2. 降低长期写入空间消耗（空间）。
3. 支持多进程共享元数据与安全初始化（并发）。
4. 对齐嵌入式可部署需求（工程可落地）。

3.2 事件格式与元数据结构

当前 Optbinlog 采用 “共享元数据文件 + 进程内共享视图缓存 + 自描述二进制帧 + `CRC32C(hw)` + string-heavy 场景自动 `varstr`” 的实现方式。

事件定义仍采用 `(name|type|bits)` 形式，字段类型简化为 `L/D/S`：

1. `L`：定长整数，按 schema 指定字节数写入。
2. `D`：双精度浮点，固定写入 8 字节。
3. `S`：字符串字段；当前实现同时支持固定宽度字符串编码和 `varstr` 编码。

共享元数据文件内仍由以下结构组织：

1. `OptbinlogSharedTag`：共享文件头与初始化状态。
2. `OptbinlogBitmap`：tag 合法性位图。
3. `OptbinlogEventTag`：事件定义入口。
4. `OptbinlogEventTagEle`：元素类型与长度描述。

在共享文件之上，系统进一步引入了**进程内共享视图缓存层**。该缓存按共享文件的 `dev/inode/size/mtime` 判定是否可复用，并缓存：

1. 已映射的共享文件视图。
2. 每个 tag 的 `OptbinlogTagCacheEntry`。
3. schema 级统计量，例如 tag 数、总 payload 字节数、字符串字段总宽度、字符串字段数和最大字符串字段宽度。

这些统计量会直接参与写路径中的 `varstr` 自动决策，因此它们属于当前热路径优化的一部分。

3.3 写入/读取路径

写入路径：1. 通过 `acquire_shared_view()` 获取共享文件视图；若共享文件未变化，则直接复用进程内缓存的 tag cache。2. 根据 schema 统计量判断是否启用 `varstr`；当字符串字段占比高或最大定长字符串字段较大时，默认切换到 `varstr`。3. 依据 tag cache 顺序编码 payload。4. 构造 4 字节自描述帧头，其中同时编码 `payload_len`、`varstr` 标志位和校验类型。5. 对单条记录计算校验值；默认采用 `CRC32C(hw)`，仅在硬件不可用时回退到软件实现。6. 将 `frame_header + payload + checksum` 写入用户态缓冲区，按批次落盘。

读取路径：1. 读取 4 字节帧头，并解析 `payload_len`、`varstr` 标志位与 `checksum_type`。2. 读取 payload 与校验字段，并按帧头声明的校验类型执行校验。3. 按共享格式解码：整数和浮点按固定宽度读取，字符串则根据 `varstr` 标志选择 “长度前缀 + 数据” 或 “固定宽度截断/补零” 方式。4. 将解码后的记录交给统一回调接口。

因此，当前记录格式可写作：

$$\text{frame} = \text{frame_header} \parallel \text{payload} \parallel \text{checksum}$$

其中：

$$\text{frame_header} = \text{payload_len} \cup \text{varstr_bit} \cup \text{checksum_type}$$

3.4 关键迭代

1. 从 “每事件独立结构” 迭代为 “统一结构 + 位图 + 偏移表”。
2. 从进程内指针共享迭代为 “偏移量共享”（解决跨进程映射基址差异）。
3. 从 switch-case 查找迭代为 `arr=id/100, idx=id%100` 的常数级路径。
4. 从软件 `CRC32` 迭代为默认 `CRC32C(hw)`；在 ARMv8 与 SSE4.2 平台优先使用硬件指令，软件实现只作为回退路径。
5. 新增自描述帧头，使 `CRC32 / CRC32C / none` 与 `fixed-string / varstr` 可以在同一读取逻辑中兼容。
6. 新增进程内共享视图缓存和 tag cache，把共享文件 reopen、tag 遍历与 schema 统计从稳态热路径迁出。
7. 新增 schema 级自动 `varstr` 策略：当字符串字段占比高时，从固定宽度字符串编码切换到 “长度前缀 + 实际长度” 编码。
8. 增加并发初始化状态机和 init-race 压力验证，保证共享文件只被单创建者初始化，其余进程或节点复用既有元数据。

从实现逻辑上看，最关键的三条设计收敛点是：

1. **校验语义与热路径代价兼顾**：系统保留每条记录校验，同时通过 `CRC32C(hw)` 将校验成本压缩到更适合热路径的水平。
2. **共享元数据的稳态收益进入真实路径**：共享视图缓存和 tag cache 使 schema 解析与 tag 遍历不再重复进入每次写入。

3. **字符串主导 schema 的冗余写入被抑制**：在 string-heavy 场景下自动启用 `varstr`，把固定宽度补零和无效拷贝降到更低。

3.5 与实际场景的匹配性

1. 小样本 `N`：共享文件打开、共享视图校验、文件打开和缓冲准备等固定项占比仍然较高，时间优势不一定立刻出现。
2. 中大样本 `N`：固定项被摊薄，自描述帧头和 `CRC32C(hw)` 的额外代价收敛到常数级，binary 的时间和空间优势更稳定。
3. string-heavy schema：自动 `varstr` 会显著减少补零写入和冗余字节，因此空间优势更容易显现。
4. 高并发 `D`：即使校验和 schema 解析成本被压低，调度、文件竞争和写出路径仍会主导吞吐峰值与回落点。
5. 多进程/多节点部署：共享 schema、进程内缓存复用和节点级初始化竞争正确性已经形成闭环，因此该设计更贴近真实工程部署。

---

4 理论模型与数学推导

本章先做抽象推导，不代入具体事件文件数值。

4.1 符号定义

1. N：总日志条数。
2. D：并发设备数（或并发进程数）。
3. r：每设备日志条数，满足 $N = D \cdot r$。
4. k：事件类型（tag）索引，$k \in \{1, \ldots, T\}$。
5. p_k：事件类型 k 的概率，满足 $\sum_k p_k = 1$。
6. e_k：事件 k 的字段数。
7. $l_{k,j}$：事件 k 第 j 个字段编码字节数。
8. $L_k = \sum_{j=1}^{e_k} l_{k,j}$：事件 k 的有效载荷字节数。
9. A_m：模式 m 的固定时间开销（初始化、打开文件、预热）。
10. C_m：模式 m 的单条边际时间开销。

11. B_m：模式 m 的单条边际空间开销。
12. $S_{0,m}$：模式 m 的固定空间开销（例如共享元数据）。
13. Q_m：吞吐（records/s）。

4.2 从算法与数据结构到单条代价

为避免“只看结果不看机制”，先把每种模式拆成操作级成本。

4.2.1 text

单条记录包含：1. 读取输入字段。2. `snprintf`/拼接格式串。3. `write` 到文件。

时间可写为：

$$t_{\text{text}}(k) = t_{\text{parse}}(k) + t_{\text{fmt}}(k) + t_{\text{write}}(\text{bytes}_{\text{text}}(k))$$

空间可写为：

$$b_{\text{text}}(k) = b_{\text{prefix}} + b_{\text{delim}} + b_{\text{num}}(k) + b_{\text{str}}(k)$$

其中 $b_{\text{num}}(k)$ 是数字转十进制后的字符长度，随数值大小缓慢变化。

4.2.2 binary（optbinlog）

单条记录包含：1. tag 查找（数组/偏移访问）。2. 按字段类型编码 payload。3. 计算校验值。4. 写帧 `len + payload + crc`。

时间：

$$t_{\text{bin}}(k) = t_{\text{lookup}} + t_{\text{encode}}(k) + t_{\text{checksum}}(L_k) + t_{\text{write}}(4 + h_{\text{bin}} + L_k + 4)$$

空间：

$$b_{\text{bin}}(k) = 4 + h_{\text{bin}} + L_k + 4$$

固定空间：

$$S_{0,\text{bin}} = S_{\text{shared}}$$

若 schema 触发 `varstr`，则字符串字段的有效载荷不再按固定宽度写出，而是写为“长度前缀 + 实际长度”：

$$L_k^{(\text{varstr})} = \sum_{j \in \mathcal{N}_k} l_{k,j} \; + \sum_{j \in \mathcal{S}_k} \left(2 + \hat{l}_{k,j}\right), \qquad \hat{l}_{k,j} \le l_{k,j}$$

4.2.3 syslog / ftrace

抽象为“系统日志接口 + 内核路径”：

$$t_{\text{sys}}(k) = t_{\text{format_sys}}(k) + t_{\text{syscall}} + t_{\text{kernel_path}} + t_{\text{write_kernel}}$$

$$t_{\text{ftrace}}(k) = t_{\text{format_ftrace}}(k) + t_{\text{trace_write}} + t_{\text{ring_or_file_path}}$$

空间：

$$b_{\text{sys}}(k) = b_{\text{sys_header}} + b_{\text{sys_payload}}(k)$$

$$b_{\text{ftrace}}(k) = b_{\text{ftrace_header}} + b_{\text{ftrace_payload}}(k)$$

4.2.4 nanolog_like / zephyr_deferred_like / ulog_async_like / hilog_lite_like

`nanolog_like`（固定字段二进制）：

$$t_{\text{nano}} = t_{\text{pack_fixed}} + t_{\text{memcpy}} + t_{\text{write}}$$

$$b_{\text{nano}} = \text{const}$$

`zephyr_deferred_like`（前台入队 + 后台批刷）：

$$t_{z,\text{front}} = t_{\text{enqueue}} + t_{\text{signal}}$$

$$t_{z,\text{back}} = \frac{t_{\text{dequeue}} + t_{\text{batch_write}}(k_b)}{k_b}$$

$$t_z = t_{z,\text{front}} + t_{z,\text{back}}$$

$$b_z = b_{\text{queue_node}} + b_{\text{fixed}} + \frac{b_{\text{batch_meta}}}{k_b}$$

`ulog_async_like`（异步队列 + 文本 backend）：

$$t_{\text{ulog}} = t_{\text{enqueue}} + t_{\text{cond_signal}} + \frac{t_{\text{dequeue}} + t_{\text{format_text}} + t_{\text{backend_write}}}{k_b}$$

$$b_{\text{ulog}} = b_{\text{queue_node}} + b_{\text{text_payload}}$$

`hilog_lite_like`（受限字段 + 轻量头）：

$$t_{\text{hilog}} = t_{\text{compact_format}} + t_{\text{write}}(h_{\text{lite}} + p_{\text{lite}})$$

$$b_{\text{hilog}} = b_{\text{header_lite}} + b_{\text{payload_lite}}$$

4.3 空间随 N 的详细推导

4.3.1 期望单条字节

对任意模式 m，定义：

$$B_m = \mathbb{E}[b_m(k)] = \sum_k p_k\, b_m(k)$$

则总空间：

$$S_m(N) = S_{0,m} + NB_m$$

这是严格的一阶线性模型，斜率为 B_m，截距为 $S_{0,m}$。

4.3.2 binary 的显式表达

由 4.2.2 得：

$$b_{\text{bin}}(k) = 8 + h_{\text{bin}} + L_k$$

$$B_{\text{bin}} = 8 + h_{\text{bin}} + \sum_k p_k L_k$$

$$S_{\text{bin}}(N) = S_{\text{shared}} + N\left(8 + h_{\text{bin}} + \sum_k p_k L_k\right)$$

4.3.3 节省率与导数

binary 相对模式 x 的空间节省率：

$$\eta_{\text{bin}/x}(N) = 1 - \frac{S_{\text{bin}}(N)}{S_x(N)}$$

若 $S_{0,x} = 0$，则

$$\eta_{\text{bin}/x}(N) = 1 - \frac{B_{\text{bin}} + S_{\text{shared}}/N}{B_x}$$

求导：

$$\frac{d\eta_{\text{bin}/x}}{dN} = \frac{S_{\text{shared}}}{N^2 B_x} > 0$$

说明 N 越大，S_{shared}/N 越小，binary 空间优势越稳定。

4.3.4 空间交叉点

当比较 bin 与 x：

$$S_{\text{bin}}(N) \leq S_x(N)$$

$$S_{\text{shared}} + NB_{\text{bin}} \leq S_{0,x} + NB_x$$

若 $B_x > B_{\text{bin}}$，可得交叉点：

$$N^*_{\text{space}}(\text{bin}, x) = \frac{S_{\text{shared}} - S_{0,x}}{B_x - B_{\text{bin}}}$$

当 $N > N^*_{\text{space}}$ 时，binary 空间一定优于 x。

4.4 时间随 N 的详细推导

4.4.1 从单条时间到总时间

定义：

$$C_m = \mathbb{E}[t_m(k)] = \sum_k p_k\, t_m(k)$$

则总时间：

$$T_m(N) = A_m + NC_m$$

该形式直接对应实验中的 `elapsed/write_only/end_to_end` 指标。

4.4.2 吞吐公式与单调性

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N}$$

对 N 求导：

$$\frac{dQ_m}{dN} = \frac{1000A_m}{N^2\left(C_m + A_m/N\right)^2} > 0$$

说明在模型中吞吐随 N 单调上升并渐近：

$$\lim_{N\to\infty} Q_m(N) = \frac{1000}{C_m}$$

二阶导在常见参数下为负，表示“先快后慢”的收益递减，这与实验中大 N 后增益变缓一致。

4.4.3 时间交叉点（binary 何时优于其他模式）

比较 bin 与 x：

$$T_{\text{bin}}(N) \leq T_x(N)$$

$$A_{\text{bin}} + NC_{\text{bin}} \leq A_x + NC_x$$

当 $C_x > C_{\text{bin}}$ 时：

$$N^*_{\text{time}}(\text{bin}, x) = \frac{A_{\text{bin}} - A_x}{C_x - C_{\text{bin}}}$$

解释：1. 若 A_{bin} 较大（共享初始化重），小 N 可能不占优。2. 随 N 增大，NC 项主导，只要 $C_{\text{bin}} < C_x$，最终会跨过交叉点。

4.4.4 C_m 的字段复杂度展开

为把 “常数项差异” 写清楚，可把边际开销展开到字段数级别：

$$C_m = c_{m,0} + c_{m,1}\mathbb{E}[e_k] + c_{m,2}\mathbb{E}[L_k] + c_{m,3}\mathbb{E}[\text{syscall}_m]$$

其中：1. $\mathbb{E}[e_k]$ 决定字段遍历成本（编码循环次数）。2. $\mathbb{E}[L_k]$ 决定 memcpy、校验和写入字节相关成本。3. $\mathbb{E}[\text{syscall}_m]$ 反映模式是否强依赖系统调用路径。

可得典型关系：1. text/syslog：$c_{m,1}$ 与 $c_{m,3}$ 较大（格式化 + syscall）。2. binary：$c_{m,1}$ 中等，$c_{m,2}$ 受字段编码与校验影响，另有 A_{bin} 较大（共享文件和缓存准备）。3. nanolog_like：$\mathbb{E}[e_k]$ 近似常数且小，$\mathbb{E}[\text{syscall}_m]$ 低，C_m 最小。4. zephyr_deferred_like：前台 C 小，但高并发时等待项会转移到 $W_m(D, r)$。5. ulog_async_like：前台 C 小，但后台文本格式化与后端写入使 $W_m(D, r)$ 与 C_m 的常数项上升。6. hilog_lite_like：通过限制格式能力降低 $\mathbb{E}[e_k]$ 与 $\mathbb{E}[L_k]$，通常较 text/syslog 更轻。

4.5 多设备 D 的详细推导

4.5.1 总时间分解

设每设备写 r 条：

$$E_m(D, r) = A_{\text{spawn}}(D) + A_m + rC_m + W_m(D, r)$$

其中 W_m 为竞争等待项（锁、队列、内核资源、I/O 饱和）。

4.5.2 队列化近似（解释吞吐饱和）

把共享瓶颈抽象成单服务台，服务率 μ_m（records/s），到达率 $\lambda = D\nu$（ν 为单设备提交率）。

稳定条件：

$$\rho = \frac{\lambda}{\mu_m} < 1$$

等待时间近似随 ρ 增大而快速上升，可抽象为：

$$W_m(D, r) \sim r f_m(\rho), \qquad f_m'(\rho) > 0, \qquad f_m''(\rho) > 0$$

因此当 D 接近

$$D_{\text{sat},m} \approx \frac{\mu_m}{\nu}$$

吞吐进入平台区；超过后出现明显回落。

4.5.3 常用二次近似

在工程拟合中常写作：

$$E_m(D, r) \approx \beta_m + rC_m + \alpha_m D + \gamma_m D^2$$

总吞吐：

$$Q_m(D, r) = \frac{1000Dr}{E_m(D, r)}$$

当 $\gamma_m > 0$ 时，D 过大必然导致分母增长过快，出现峰值后下降。

进一步对 D 求导：

$$\frac{dQ_m}{dD} \propto \left(\beta_m + rC_m + \alpha_m D + \gamma_m D^2\right) - D\left(\alpha_m + 2\gamma_m D\right)$$

整理得：

$$\frac{dQ_m}{dD} \propto \beta_m + rC_m - \gamma_m D^2$$

故吞吐峰值点近似为：

$$D_{\text{peak},m} = \sqrt{\frac{\beta_m + rC_m}{\gamma_m}}$$

这给出了可直接与实验扫描结果比对的“理论峰值设备数”。

4.5.4 空间随 D

$$S_m(D, r) = S_{0,m} + DrB_m$$

因此空间对 D 严格线性，差异主要由斜率 B_m 与截距 $S_{0,m}$ 决定。

4.6 模式间理论排序与可检验预测

4.6.1 异语义比较（最短路径口径）

由单条代价可得典型关系：

$$B_{\text{nano}} \sim B_z \lesssim B_{\text{hilog}} < B_{\text{bin}} < B_{\text{sys}} \sim B_{\text{text}} \lesssim B_{\text{ulog}}$$

C_{nano} 常最小，C_{hilog} 次之；C_z 与 C_{ulog} 受队列和后台行为影响；$C_{\text{text}}/C_{\text{sys}}/C_{\text{ftrace}}$ 受格式化与系统路径影响更大。

可检验预测：1. nanolog_like 在时间与吞吐上常呈数量级优势。2. hilog_lite_like 在“受限语义”下应明显优于 text/syslog。3. ulog_async_like 在吞吐未饱和且后台轻负载时有机会受益；若后台文本后端成为瓶颈，可能慢于 binary/text。4. 上述优势不代表语义等价下仍保持同倍率。

4.6.2 语义对齐比较（公平口径）

当统一为 `len + payload + crc` 并统一字段集时：

$$B_{\text{sem}}(\text{bin}) \approx B_{\text{sem}}(\text{nano}) \approx B_{\text{sem}}(\text{zephyr})$$

C_{sem} 的差异主要来自实现常数（编码循环、队列同步、缓存命中和校验实

现），不再是语义体量差异。

可检验预测：1. 空间差异收敛到小常数级。2. 时间差异收敛到同一量级，而非数量级差异。

4.7 与第 5 章指标的映射关系

为了让推导可被实验验证，建立公式到指标的映射：

1. $T_m(N)$ 对应 `elapsed_ms` 或 `end_to_end_ms` 的均值趋势。
2. Q_m 对应 `throughput_rps` / `throughput_e2e_rps`。
3. S_m 对应 `total_bytes`。
4. N^*_{time} 与 N^*_{space} 对应 “某场景下 binary 由劣转优” 的阈值。
5. $D_{\text{sat},m}$ 对应 “多设备扫描中吞吐峰值后回落” 的拐点。

本章给出的是 “由算法与数据结构直接推出的理论趋势”；第 5 章再代入实测数据检验这些趋势是否成立，以及偏离原因。

---

5 实验迭代、对比与理论一致性分析

5.1 实验组织与迭代逻辑

本章采用 “先语义对齐，再模式对照” 的方法，避免将语义体量差异误判为实现差异。实验主线分为两层：工程近似层使用 2026-03-09 的 grouped semantic matrix 运行，主要回答 “真实系统路径代价如何排序”；严格公平层使用 2026-03-12 的 final aligned suite 运行，统一以 `text_semantic_like / binary / peer semantic_like` 为比较对象，覆盖以下三类场景：1. 单机高负载(single)。2. 本地多设备模拟（`D=2,5,10` 扫描）。3. 模拟真实多设备（多台虚拟机节点）并发（10 节点，每节点一台设备）。

此外，本文同步更新了两组初始化竞争结果：本地多进程初始化运行和节点级初始化运行。统计口径统一为 `mean/median/p95/std/95%CI`，先执行 IQR（1.5）异常值过滤，再输出时间、空间、吞吐及改进百分比，以保证各组实验可横向比较。

实验数据按用途分成两层。2026-03-09 的 `grouped_semantic_matrix` 主要承担“工程近似对照”角色：`syslog/ftrace` 组的输入与系统路径较接近真实工程，适合观察系统级路径代价；2026-03-12 的严格语义对齐套件统一使用 `text_semantic_like / binary / peer semantic_like`，覆盖 single、local multi-device 和 10 节点真实多设备三类场景，作为本文讨论 `nanolog/zephyr/ulog/hilog` 四组公平结论的主体证据。消融结果只用于解释原因，不承担主结论角色。

5.2 单机高负载：严格语义对齐结果

本节结果来自严格语义对齐套件的合并汇总文件。三类模式统一采用同一 `eventlogst_semantic_*` schema、同一生成器产生的字段值，以及同一平台；其中 `binary` 采用“共享 schema/tag cache + per-record CRC32C(hw) + string-heavy auto-varstr”的当前实现。

表 5-1 单机高负载下 binary 相对 text 的结果

分组	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%	结论
nanolog	+92.13%	+69.79%	+1171.80%	相对文本路径收益明显
zephyr	+94.12%	+72.59%	+1601.58%	相对文本路径收益明显
ulog	+76.20%	+43.35%	+315.72%	相对文本路径收益明显
hilog	+84.07%	+53.87%	+527.74%	相对文本路径收益明显

表 5-2 单机高负载下 binary 相对 peer 的结果

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	结论
nanolog	+11.13%	-30.85%	+10.51%	binary 在时间上略快于最短紧凑二进制 peer，但空间仍更大
zephyr	-12.82%	-28.66%	-11.19%	deferred/batch 热路径仍以更短的记录组织保留时间优势
ulog	+65.78%	+33.50%	+192.21%	binary 明显优于文本扩展更重的 peer 路径
hilog	+78.55%	+48.51%	+366.23%	binary 在时间、空间与吞吐三项上均显著占优

如表 ~5-1 和表 ~5-2 所示，该结果体现出两层清晰结构。第一，`binary vs text_semantic_like` 在四组上全部为正，说明一旦输入语义完全一致，文本格式化与字符串拼接的代价会稳定拉高 `text` 路径。第二，`binary vs peer semantic_like` 呈现两类分化：`ulog/hilog` 属于“文本扩展更重”的语义组，binary 的结构化编码优势直接转化为时间、空间和吞吐优势；`nanolog/zephyr` 则更接近“最短二进制热路径”或“deferred 批处理热路径”，因此 binary 的通用字段遍历与帧组织常数项会更加显性。

因此，工程近似层与严格公平层需要分别承担不同问题：前者回答“真实

系统路径大致如何排序”，后者回答“在同一 schema、同一字段值、同一平台下，Optbinlog 相对 peer 的本征开销是什么”。

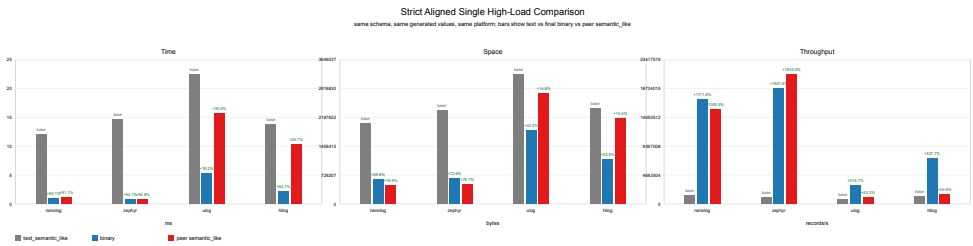

图 5-1 严格语义对齐单机高负载总览图

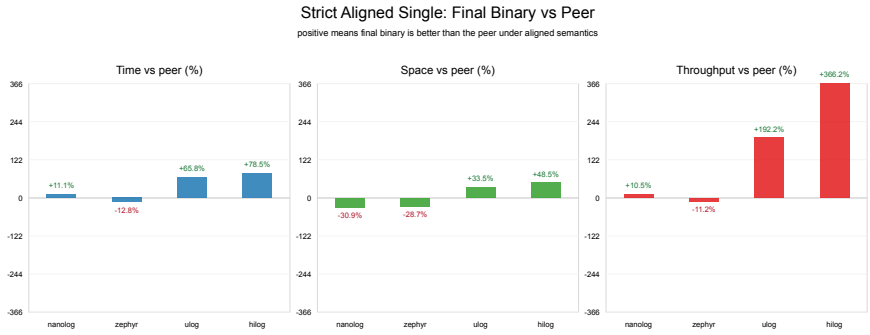

图 5-2 严格语义对齐单机场景中 binary 相对 peer 的差值图

5.3 本地多设备模拟：竞争项放大后的趋势

本节结果同样来自 `final_aligned_summary.json`，但切换到 `multi` 部分，并对 `D=2,5,10` 进行扫描。该场景保持平台不变，只让设备数增加，因此能够单独观察 `W_m(D,r)` 对趋势的影响。

表 5-3 本地多设备模拟下 binary 相对 peer 的扫描结果

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
nanolog	2	-6.22%	-6.49%	-31.82%	竞争项放大后，小优势先被压掉

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
nanolog	5	-52.65%	-23.56%	-31.19%	局部最差点位，说明最短热路径对进程竞争最敏感
nanolog	10	+2.47%	+2.16%	-30.98%	在更高并发下回到近似持平
zephyr	2	+9.94%	+10.46%	-29.65%	低并发下 binary 仍可略优
zephyr	5	-0.90%	-1.74%	-29.00%	中负载附近接近交叉点
zephyr	10	+1.55%	+3.09%	-28.79%	高并发下仍基本处于同一量级
ulog	2	+1.04%	+1.04%	+33.30%	相对 peer 已转入稳定正区间
ulog	5	+12.22%	+14.22%	+33.42%	竞争项放大后优势反而更清晰

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
ulog	10	+15.40%	+17.39%	+33.46%	对文本扩展型 peer 保持稳定领先
hilog	2	+14.12%	+13.79%	+48.28%	低并发即显著领先
hilog	5	+20.16%	+25.29%	+48.43%	中负载优势进一步扩大
hilog	10	+5.73%	+5.08%	+48.48%	高并发下优势被压缩但未翻转

如表 ~5-3 所示，从本地多设备扫描可以看到三类不同的曲线形态。`ulog/hilog` 组在 `D=2,5,10` 上始终保持对 peer 的正收益，说明当 peer 路径必须持续做文本扩展时，竞争项会进一步放大其格式化与写出负担；`zephyr` 组则长期围绕零附近波动，说明 deferred/batch 的摊销能力与 binary 的结构化写出已经非常接近；`nanolog` 组最敏感，`D=5` 处出现明显负跳变，到了 `D=10` 又回到近似持平，这表明对于“热路径极短、空间极紧凑”的 peer，进程调度顺序、共享文件竞争和 run-phase 对齐方式都足以改变排序。

与 `text_semantic_like` 的比较也支持这一点：除 `nanolog` 在 `D=2,5` 的局部点位外，其余大多数分组上 binary 对 text 都维持正收益。也就是说，多设备场景下真正容易把 binary 拉回负区间的，并不是“文本路径”，而是那些已经把语义压缩到极短热路径的 peer 二进制实现。

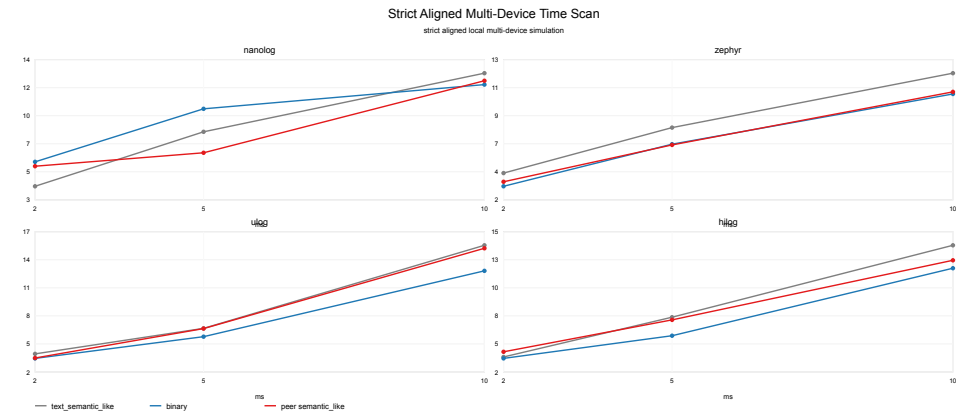

图 5-3 严格语义对齐本地多设备时间扫描图

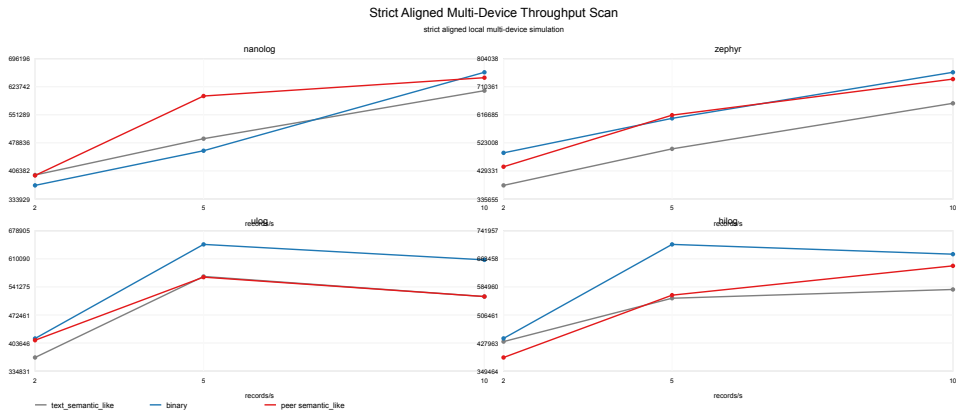

图 5-4 严格语义对齐本地多设备吞吐扫描图

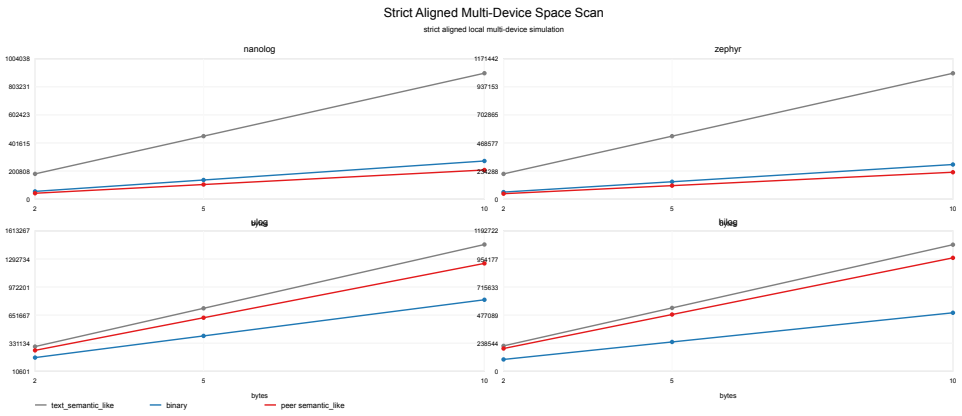

图 5-5 严格语义对齐本地多设备空间扫描图

5.4 模拟真实多设备（多台虚拟机节点）并发：严格语义对齐结果

本节结果来自严格对齐套件的 l1 节点结果及其合并汇总，采用 10 个并发节点，每节点一台设备。与本地多设备不同，这一场景在保持语义完全对齐的同

时，引入了节点级冷启动、VM 调度噪声和跨节点执行偏移，因此更接近论文中的“工程部署代理场景”。

表 5-4 真实多设备模拟下 binary 相对 text 的结果

分组	节点有效数	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%
nanolog	10/10	+36.97%	+69.79%	+72.89%
zephyr	10/10	+58.75%	+72.59%	+182.17%
ulog	10/10	+56.52%	+43.35%	+174.61%
hilog	10/10	+62.52%	+53.87%	+205.85%

表 5-5 真实多设备模拟下 binary 相对 peer 的结果

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%
nanolog	-8.21%	-30.85%	-19.48%
zephyr	-79.33%	-28.66%	-62.52%
ulog	+43.94%	+33.50%	+99.36%
hilog	+53.95%	+48.51%	+152.27%

如表 ~5-4 和表 ~5-5 所示，真实多设备模拟的结论与单机、本地多设备形成了更完整的三层证据链。首先，binary 相对 `text_semantic_like` 在四组上全部保持明显正收益，说明在节点级并发下，文本格式化仍然是稳定的高开销基线。其次，`ulog/hilog` 组在真实多设备下仍显著领先 peer，说明其主要矛盾仍是“文本展开和较长记录组织”；节点噪声会压缩优势，但不会改变方向。再次，`nanolog/zephyr` 两组则表现出更强的边界性：`nanolog` 从单机的 `+11.13%` 退回到真实多设备的 `-8.21%`，`zephyr` 从单机的 `-12.82%` 进一步扩大到 `-79.33%`。这说明当 peer 本身已经是极短二进制热路径或 deferred 批处理路径时，节点级冷启动、每节点单独创建进程、缓存收益难以完全释放等因素，会让 binary 的通用路径常数项重新显性化。

因此，“真实多设备下谁更优”并不是单机结论的简单复制，而是对第 4 章模型中 `W_m(D,r)` 与冷路径项 `A_m` 的联合验证：`ulog/hilog` 由文本扩展主导，binary 仍能稳定获益；`nanolog/zephyr` 则更受热路径长度和摊销粒度控制，因而更容易在节点级并发下失去单机上的微弱优势。

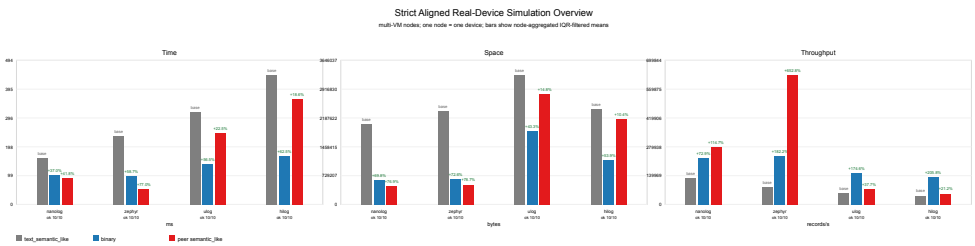

图 5-6 严格语义对齐真实多设备总览图

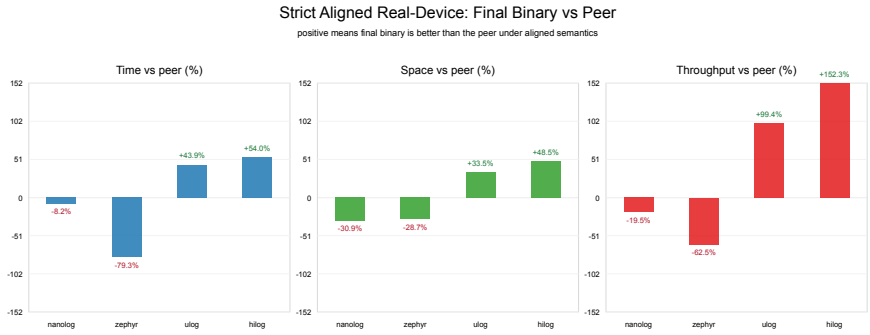

图 5-7 严格语义对齐真实多设备场景中 binary 相对 peer 的差值图

5.5 初始化竞争说明

初始化竞争结果同步更新如下：1. 本地多进程初始化竞争。

elapsed_mean = 14.173 ms，p95 = 16.804 ms，create_success_mean = 1.0，open_existing_ok_mean = 9.0。

2. 模拟真实多设备（多台虚拟机节点）初始化竞争。

global_elapsed_mean = 6419.830 ms，p95 = 6550.884 ms，single_creator_mean = 1.0，failed_nodes_mean = 0.0。

两组结果均稳定满足“单创建者、其余节点复用已初始化共享文件”的正确性目标。

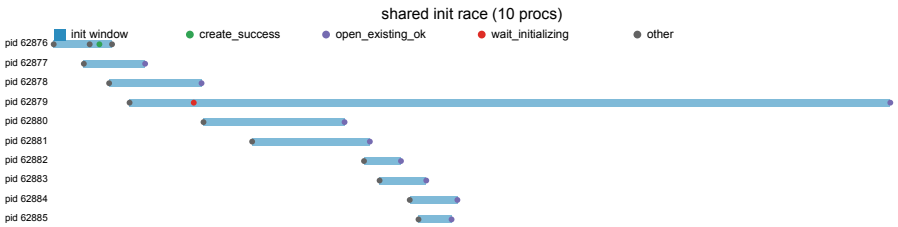

图 5-8 本地多进程初始化竞争时序图

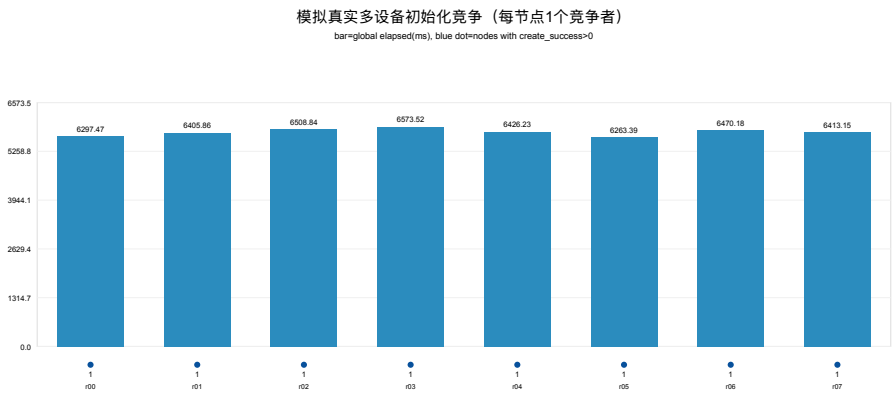

图 5-9 真实多设备节点级初始化竞争总览图

5.6 理论推导与实测对照

综合 5.2~5.4 与第 4 章模型，可得到四点结论：

1. 空间模型与实测一致：`total_bytes` 随 `N` 线性增长，而不同语义组对应不同 `B_m`。因此在 `nanolog/zephyr` 组中，binary 相对 peer 仍保留约 `28%~31%` 的空间劣势；而在 `ulog/hilog` 组中，binary 则持续节省约 `33%~49%` 的空间。
2. 时间交叉点是“按语义组出现”的，而不是“按模式统一出现”的。`ulog/hilog` 在 single、local multi 和 real multi-node 三层场景中都保持对 peer 的正收益；`zephyr` 则长期位于负区或零附近；`nanolog` 则呈现单机略正、多设备局部反转、真实多设备小幅转负的边界形态。
3. 竞争项 `W_m(D,r)` 在本地多设备与真实多设备场景中都可被直接观察到。`nanolog` 在 `D=5` 的负跳变、`hilog` 在 `D=10` 的优势压缩，以及 `zephyr` 在 L1 场景中的显著回落，都是该竞争项和冷路径项共同放大的体现。
4. “真实多设备”不是简单复制“本地多设备”。L1 场景额外引入了节点级冷启动、VM 调度噪声和每节点新进程启动，因此 `A_m` 的占比会明显抬升；这正是 `nanolog/zephyr` 更容易在 L1 场景下失去单机优势，而 `ulog/hilog` 仍能保持正收益的主要原因。

5.7 原因分析与消融佐证

为了更直接解释上述三层场景的排序差异，本文继续使用严格公平消融运行作为“原因分析层”，但不再把它作为主体结论数据集。该数据集给出的关键消融结果如下：

表 5-6 消融实验中的冷路径与校验影响

分组	binary 冷路径 prep(ms)	`binary_nocrc` 时间改善%
nanolog	0.416	+8.88%
zephyr	0.496	+2.38%
ulog	4.258	-2.70%
hilog	1.722	+11.77%

表 5-7 消融实验中的字符串编码影响

分组	`binary_varstr` 时间改善%	`binary_nocrc_varstr` 时间改善%
nanolog	-0.15%	+0.98%
zephyr	-3.03%	-0.50%
ulog	+0.84%	-1.34%
hilog	+10.72%	+12.01%

如表 ~5-6 和表 ~5-7 所示，结合严格对齐结果与消融结果，可以把原因归纳为四点：1. **校验已经不是主导项**：`binary_nocrc` 的额外收益仅落在噪声级到 `11.77%` 区间，说明当前实现中每条记录校验的热路径成本已经被压缩到可控范围内。2. **剩余瓶颈更偏冷路径与通用打包**：`ulog/hilog` 的 `prep_ms` 仍分别达到 `4.258ms` 与 `1.722ms`。这解释了为什么 binary 在 `ulog/hilog` 这类文本扩展型 peer 面前依然有明显优势，但在 `nanolog/zephyr` 这类极短热路径面前，冷路径和 frame 组织开销会被放大。3. **auto-varstr 已成为日常路径的一部分**：`binary_varstr` 在 `ulog/hilog` 组几乎不再额外改善体积，说明 string-heavy schema 下的变长字符串收益已经被当前实现吸收，并直接体现在默认写入路径中。4. **L1 场景的反转是可解释的**：本轮严格对齐套件采用“新进程 + 单次写入”统计口径，因此 shared schema/tag cache 的稳态收益不会在每节点冷启动场景中被完全释放。于是 `nanolog` 这类单机上略占优势的组，在真实多设备场景就可能被重新压回负区；而 `ulog/hilog` 因为主要优势来自“避免文本展开”，所以即使在 L1 场景仍能维持明显正收益。

5.8 实验数据组织与证据链

为了让实验结论既能覆盖真实路径，又能满足严格公平对照，本文将数据集按用途组织为三类：1. **工程近似层**：保留 `syslog/ftrace` 以及 `*_like` 路径，观察真实工程风格系统的整体代价排序。2. **严格公平层**：统一使用 `text_semantic_like / binary / peer semantic_like`，在相同 schema、相同字段值和相同平台上报告单机、本地多设备与真实多设备结果。3. **原因分析层**：保留 `binary_hotpath / binary_nocrc / binary_varstr` 等消融模式，用于解释当前实现中的主要时间来源和空间来源。

其中，工程近似层负责回答“真实系统路径如何排序”，严格公平层负责回答“当前实现与 peer 在同语义下如何分化”，原因分析层负责回答“差异主要来自哪些结构性开销”。三类数据各司其职，避免把部署差异、语义差异和实现差异混在同一张表里。

因此，论文中关于 `nanolog/zephyr/ulog/hilog` 四组的公平性结论，由 single、local multi-device 和 real multi-node 三层严格对齐结果共同支撑；工程近似层则作为工程部署视角的补充说明。

5.9 差异点原因与 Optbinlog 应用优势

结合工程近似对照、严格对齐套件和消融结果，可将 `Optbinlog` 的差异点归纳为三层：1. **文本扩展型语义组**：`ulog/hilog` 的主要矛盾是文本扩展与记录膨胀，因此 binary 在 single、local multi 和 real multi-node 三层场景中都保持稳定优势。这是 Optbinlog 当前最稳的受益区间。2. **极短热路径语义组**：`nanolog/zephyr` 已经把热路径压得很短。对于这类 peer，Optbinlog 的剩余通用开销主要来自 schema 访问、字段遍历、frame 组织和冷启动，因此单机上可以逼近甚至超过 `nanolog`，但在真实多设备场景中仍可能因为冷路径和节点噪声而回落。3. **平台与并发层**：进入真实多设备后，`W_m(D,r)` 和冷启动项会把细小常数项放大，因此“单机反超”不应被直接外推为“节点级并发反超”。这也是本文坚持同时报告 single、multi 和 L1 三层结果的原因。

Optbinlog 的应用优势主要体现在：1. **跨进程一致性**：共享 schema + 偏移访问保证多进程解码一致，适合网关/边缘设备的多服务部署。2. **工程可迁移性**：不依赖编译期代码生成链路，新增日志事件只需更新 schema 文件，适合迭

代频繁的业务系统。3. **可验证性**：二进制帧包含长度与 per-record CRC32C，损坏检测与回归测试路径清晰，便于工程质量控制。4. **对文本型 peer 的稳定优势**：`ulog/hilog` 三层场景的对齐结果表明，binary 对文本扩展更重的 peer 风格路径具有稳定时间、空间和吞吐优势。5. **优化路径清晰**：实验已经把剩余开销收敛到 `cold-path` + 通用打包路径 + 稳态缓存收益释放不足，后续优化可以围绕长生命周期进程、steady-state fast path 和批量提交策略展开。

完整分析表、最新可视化和关键结论汇总见严格对齐套件生成的报告；工程近似层的补充说明见实验补充分析报告。

---

6 结论与展望

6.1 工作总结

本文形成了“调研 -> 设计 -> 理论 -> 实验 -> 对照”的完整研究链条：

1. 调研层：明确了国内外先进日志系统的共性优化手段（deferred、ring buffer、二进制结构化、编译期迁移热路径开销）。
2. 设计层：Optbinlog 通过共享格式与偏移访问实现跨进程一致性和低冗余写入。
3. 理论层：推导了 `N` 与 `D` 对时间/空间/吞吐的关系，并给出模式差异来源。
4. 实验层：多轮结果验证了“时间优势与语义体量、并发竞争强相关”的核心结论，并完成了 NanoLog/Zephyr/uLog/HiLog Lite 与 syslog/ftrace 的分组语义对齐对比；工程近似层、严格公平层以及两类初始化竞争结果共同形成了统一证据链。

工程应用层面，Optbinlog 的价值可概括为：在保持语义可解码与 per-record CRC32C 校验能力的前提下，以共享 schema/tag cache 降低多进程与长生命周期服务的重复解析成本，并在真实多节点并发中维持稳定收益；这使其更适合“长期在线运行、版本持续演进”的嵌入式网关与边缘控制器场景。

6.2 当前局限

1. 受网络与平台限制，尚未完成外部系统官方工程的全栈编译复现（当前为代码级阅读 + 算法级原型复现）。

2. Linux 实验仍以 VM 为主，尚未覆盖 Raspberry Pi / i.MX 实机。
3. 严格公平语义套件已经扩展到 10 节点 VM 级“真实多设备”模拟，但仍未迁移到真实 ARM 板卡或 bare-metal Linux 实机。
4. 当前严格公平套件采用“新进程 + 单次写入”统计口径，因此新加入的进程内 shared/tag cache 收益只在代码层完成接入，尚未通过长生命周期 steady-state 基准充分量化。
5. 缺少 `perf` 级 CPU 微观指标，暂不能细粒度解释所有异常点。

6.3 下一步优化计划

1. `P0`：继续区分“严格公平层”和“工程近似层”。
   严格公平层统一为 `text_semantic_like / optbinlog-binary / peer semantic_like`；工程近似层保留 `syslog(journald) / ftrace` 等真实系统路径。两层结论不再混写。
2. `P0`：优先优化 `cold-path +` 通用打包路径。
   当前实现已采用 CRC32C(hw)，下一步重点不是“去掉 CRC”，而是释放进程内 schema/tag cache 的稳态收益、继续压缩记录物化和字段遍历，并沿用现有消融模式做回归。
3. `P1`：补齐平台覆盖。
   至少完成 x86_64 Linux 与 ARM Linux（Raspberry Pi / i.MX），并把严格公平语义套件原样迁移到新平台。
4. `P1`：补齐 CPU 指标。
   加入 `cycles`、`context-switch`、`cache-miss`，进一步验证 `prep`、字段遍历和打包拷贝的微观代价。
5. `P2`：扩展容错实验。
   增加损坏帧、截断帧、恢复过程和校验失败路径统计，并评估“CRC32C / 分段 CRC32C / 可选弱校验”对可靠性的影响。
6. `P1`：补齐平台仿真方案。
   在实机不足时，先用“异构 VM + `tc netem` + 不同文件系统镜像”逼近平台差异：例如将 10 节点划分为 `2C/2G` 与 `4C/8G` 两组，分别注入不同延迟/丢包，文件系统采用 `ext4/xfs` 组合；随后把同一套 `single/multi/` 模拟真实多设备脚本在该仿真环境复跑，并与实机 Raspberry Pi / i.MX 的关键点位做误差标定。该方案可先完成趋势验证，再逐步替换为真实板卡数据。

参考文献

[1] Linux Kernel Documentation. Lockless Ring Buffer Design[EB/OL]. [2026-03-09]. https://www.kernel.org/doc/html/latest/trace/ring-buffer-design.html.

[2] Linux Kernel Documentation. user_events[EB/OL]. [2026-03-09]. https://www.kernel.org/doc/html/latest/trace/user_events.html.

[3] LTTng Project. LTTng Documentation: Concepts[EB/OL]. [2026-03-09]. https://lttng.org/docs/#doc-concepts.

[4] Zephyr Project. Logging[EB/OL]. [2026-03-09]. https://docs.zephyrproject.org/latest/services/logging/index.html.

[5] Zephyr Project. Kconfig.mode[EB/OL]. [2026-03-09]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/upstream/collab-sdk-dev/subsys/logging/Kconfig.mode.

[6] Zephyr Project. log_msg.c[EB/OL]. [2026-03-09]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/main/subsys/logging/log_msg.c.

[7] systemd. Journal File Format[EB/OL]. [2026-03-09]. https://systemd.io/JOURNAL_FILE_FORMAT/.

[8] Yang S, et al. NanoLog: A Nanosecond Scale Logging System (ATC 2018)[C/OL]. [2026-03-09]. https://www.usenix.org/conference/atc18/presentation/yang-stephen.

[9] PlatformLab. NanoLogCpp17.h[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/NanoLogCpp17.h.

[10] PlatformLab. RuntimeLogger.cc[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/RuntimeLogger.cc.

[11] Android Open Source Project. event.logtags[EB/OL]. [2026-03-09]. https://android.googlesource.com/platform/system/logging/+/refs/tags/android-13.0.0_r33/logcat/event.logtags.

[12] RT-Thread. ulog 文档 [EB/OL]. [2026-03-09]. https://rt-thread.github.io/rt-thread/page_component_ulog.html.

[13] RT-Thread. ulog.c[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/RT-Thread/rt-thread/master/components/utilities/ulog/ulog.c.

[14] OpenHarmony. HiLog Lite README[EB/OL]. [2026-03-09]. https://gitee.com/openharmony/hiviewdfx_hilog_lite/blob/master/README_zh.md.

[15] Linux Kernel. ring_buffer.c[EB/OL]. [2026-03-09]. https://raw.githubusercontent.com/torvalds/linux/master/kernel/trace/ring_buffer.c.

[16] systemd. journald-server.c[EB/OL]. [2026-03-09]. https://github.com/systemd/systemd/blob/main/src/journal/journald-server.c.

[17] 武汉大学计算机学院. 2024 届毕业论文（设计）成绩的评定标准 [EB/OL]. [2026-03-09]. https://cs.whu.edu.cn/info/1055/43331.htm.

[18] 武汉大学计算机学院. 2024 年优秀学士学位论文名单公示 [EB/OL]. [2026-03-09]. https://cs.whu.edu.cn/info/1055/43341.htm.

[19] 武汉大学本科生院. 关于做好 2025 届毕业论文（设计）工作的通知 [EB/OL]. [2026-03-09]. https://uc.whu.edu.cn/info/1520/115631.htm.

[20] 武汉大学哲学学院. 2024 届优秀毕业论文名单公示 [EB/OL]. [2026-03-09]. https://philosophy.whu.edu.cn/info/1113/111001.htm.

[21] 武汉大学哲学学院. 2025 届优秀毕业论文名单公示 [EB/OL]. [2026-03-09]. https://philosophy.whu.edu.cn/info/1054/119511.htm.

[22] 武汉大学电气与自动化学院. 2025 届本科毕业设计（论文）答辩工作安排（含优秀论文公示附件）[EB/OL]. [2026-03-09]. https://eea.whu.edu.cn/info/1024/4859.htm.

[23] 武汉大学社会学院. 2024 届本科生毕业论文优秀名单公示 [EB/OL]. [2026-03-09]. https://shxy.whu.edu.cn/info/1218/77911.htm.

[24] 武汉大学社会学院. 2025 届本科生毕业论文优秀名单公示 [EB/OL]. [2026-03-09]. https://shxy.whu.edu.cn/info/1218/84911.htm.

致谢

本论文（设计）完成过程中，得到了指导教师在选题、方法、实验组织和写作规范方面的持续指导；也得到了课题组同学在实验环境、脚本复核与结果讨论方面的帮助。在此一并致以诚挚感谢。感谢学院和学校提供的学习与实验条件，感谢家人长期以来的理解与支持。

附录 A 关键复现实验命令（当前保留结果）

```
# 1) 全分组语义对齐矩阵（single/multi/模拟真实多设备）
cd demo
python3 run_grouped_semantic_matrix.py \
  --out-dir "$(pwd)/results/grouped_semantic_matrix_run_20260309_full"

# 2) 最终严格对齐套件（single/multi）
python3 run_final_aligned_suite.py \
  --skip-l1 \
  --out-dir "$(pwd)/results/final_aligned_suite_20260312_210409"

# 3) 最终严格对齐套件（真实多设备，10 节点）
```

```
zsh run_final_aligned_l1.sh \
  nanolog "$(pwd)/eventlogst_semantic_nanolog" nanolog_semantic_like \
  "$(pwd)/results/final_aligned_suite_20260312_210409/l1/nanolog"
zsh run_final_aligned_l1.sh \
  zephyr "$(pwd)/eventlogst_semantic_zephyr" zephyr_deferred_semantic_like \
  "$(pwd)/results/final_aligned_suite_20260312_210409/l1/zephyr"
zsh run_final_aligned_l1.sh \
  ulog "$(pwd)/eventlogst_semantic_ulog" ulog_semantic_like \
  "$(pwd)/results/final_aligned_suite_20260312_210409/l1/ulog"
zsh run_final_aligned_l1.sh \
  hilog "$(pwd)/eventlogst_semantic_hilog" hilog_semantic_like \
  "$(pwd)/results/final_aligned_suite_20260312_210409/l1/hilog"
python3 final_aligned_merge_l1.py \
  --summary-json "$(pwd)/results/final_aligned_suite_20260312_210409/merged/fina
  --l1-root "$(pwd)/results/final_aligned_suite_20260312_210409/l1"

# 4) 本地初始化竞争（10 进程）
clang -O2 -Wall -Wextra -std=c11 -Iinclude \
  -o optbinlog_init_race optbinlog_init_race.c \
  src/optbinlog_shared.c src/optbinlog_eventlog.c
OPTBINLOG_INIT_OUT_DIR="$(pwd)/results/init_race_run_20260309" \
OPTBINLOG_EVENTLOG_DIR="$(pwd)/eventlogst" \
OPTBINLOG_INIT_PROCS=10 OPTBINLOG_INIT_REPEATS=20 OPTBINLOG_INIT_WARMUP=3 \
python3 run_init_race.py

# 5) 模拟真实多设备节点级初始化竞争（每节点一台设备）
python3 run_l1_init_compete.py \
  --config l1_config.linux_10_all_unaligned_initrace.json \
  --tag l1_init_compete_run_20260309
```